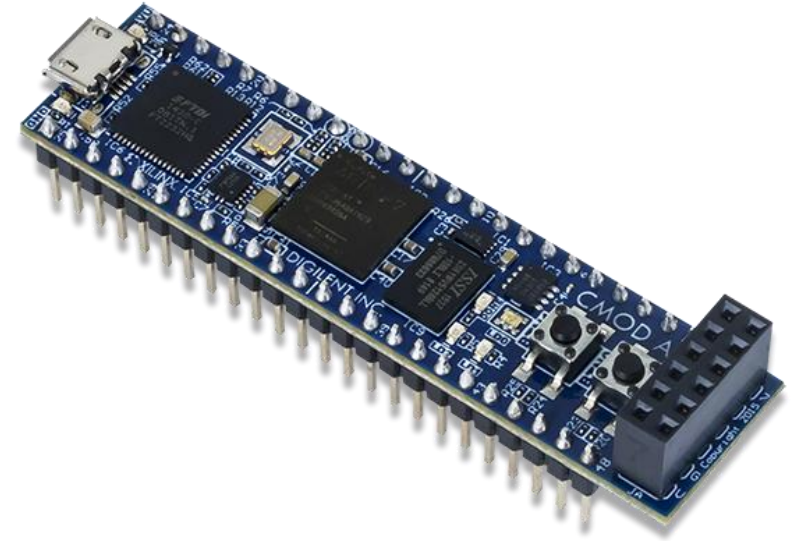


El Hanafi Nadim 894489

Informatica Industriale

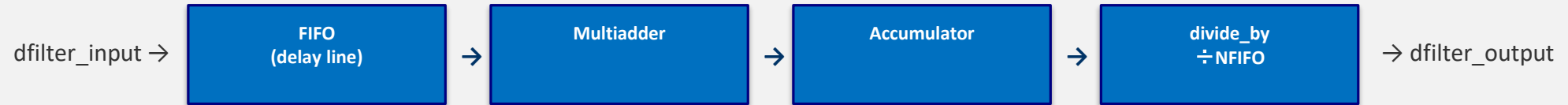


n.elhanafi@campus.unimib.it

Microelectronics Group



Introduction - filter_ma Architecture



$$y_n = y_{n-1} - d_{n-n}FIFO + d_n \quad dfilter_output = y_n / NFIFO$$

Port / Generic	Type	Width	Description
Nbf	generic	integer	Number of bits (Nb = 8)
log2NFIFO	generic	integer	$\log_2(NFIFO) \in \{5, 6, 7\}$
en / rst / clk	in	1 bit	Enable, Reset active-low, Clock 50 MHz
datain	in	Nbf bits	Input sample (signed)
dataout	out	Nbf + log2NFIFO bits	Filtered output (signed)

Nb=8 fs=50 MHz tclk=20 ns f₀=195.3 kHz OVR=128 SNR_{input}=0 dB FS=256

Multiadder — Design

```
entity multiadder is
  generic (Nb: integer; log2NFIFO: integer);
  port (
    datain_fifo, dataout_fifo:
      in  signed(Nb-1 downto 0);
    dataout_accumulator:
      in  signed(Nb+log2NFIFO-1 downto 0);
    dataout:
      out signed(Nb+log2NFIFO-1 downto 0));
end multiadder;

architecture Behavioral of multiadder is
  signal datain_fifo_large,
        dataout_fifo_large:
    signed(Nb+log2NFIFO-1 downto 0);
  signal mask0:
    signed(log2NFIFO-1 downto 0)
    := (others => '0');
begin

  datain_fifo_large <= mask0 & datain_fifo;
  dataout_fifo_large <= mask0 & dataout_fifo;

  dataout <=
    dataout_accumulator
    - dataout_fifo_large
    + datain_fifo_large;
end Behavioral;
```

Formula - Recursive MA Update

$$y_n = y_{n-1} - d_{n-nFIFO} + d_n$$

Instead of summing NFIFO samples each cycle, only the oldest sample is subtracted and the newest added. $O(1)$ complexity per clock cycle.

Signal Widths

- datain_fifo / dataout_fifo: Nb = 8 bit
- dataout_accumulator (y_{n-1}): Nb+log₂NFIFO bits
- dataout (y_n): Nb+log₂NFIFO bits

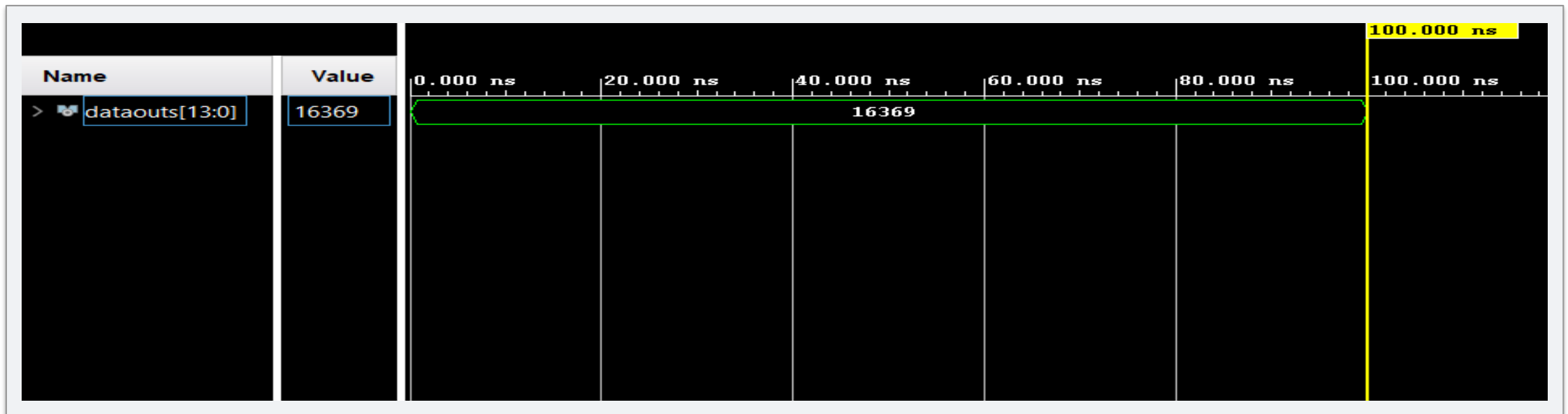
Inputs UNSIGNED → zero-extension via mask0 (no sign error).

Key Implementation Note

Samples are unsigned (0...255). Upper log₂NFIFO bits padded with '0' using mask0 concatenation. Using signed arithmetic avoids overflow and simplifies accumulator feedback.

Multiadder - Testbench

Signal	Value (dec)	Binary (14 bit)	Role
datain_fifo	1	00000000000001	d_n — current sample
dataout_fifo	15	000000000001111	d_{n-n} FIFO — oldest sample
dataout_accumulator	16383	11111111111111	y_{n-1} — running sum
dataout (expected)	16369	11111111110001	$y_n = 16383 - 15 + 1$



Accumulator - Design

```
entity accumulator is
  generic (Nb: integer; log2NFIFO: integer);
  port (
    en, rst, clk : in std_logic;
    datain : in signed(Nb+log2NFIFO-1 downto 0);
    dataout : out signed(Nb+log2NFIFO-1 downto 0));
end accumulator;

architecture Behavioral of accumulator is
  signal reg : signed(Nb+log2NFIFO-1 downto 0)
    := (others => '0');
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '0' then
        reg <= (others => '0');
      elsif en = '1' then
        reg <= datain;
      end if;
    end if;
  end process;
  dataout <= reg;
end Behavioral;
```

Function - PIPO Register

Synchronous Parallel-In Parallel-Out register.

- Stores y_n on every rising clock edge ($en='1'$)
- Output is y_{n-1} - the running sum one cycle earlier
- Feeds back to the Multiadder input

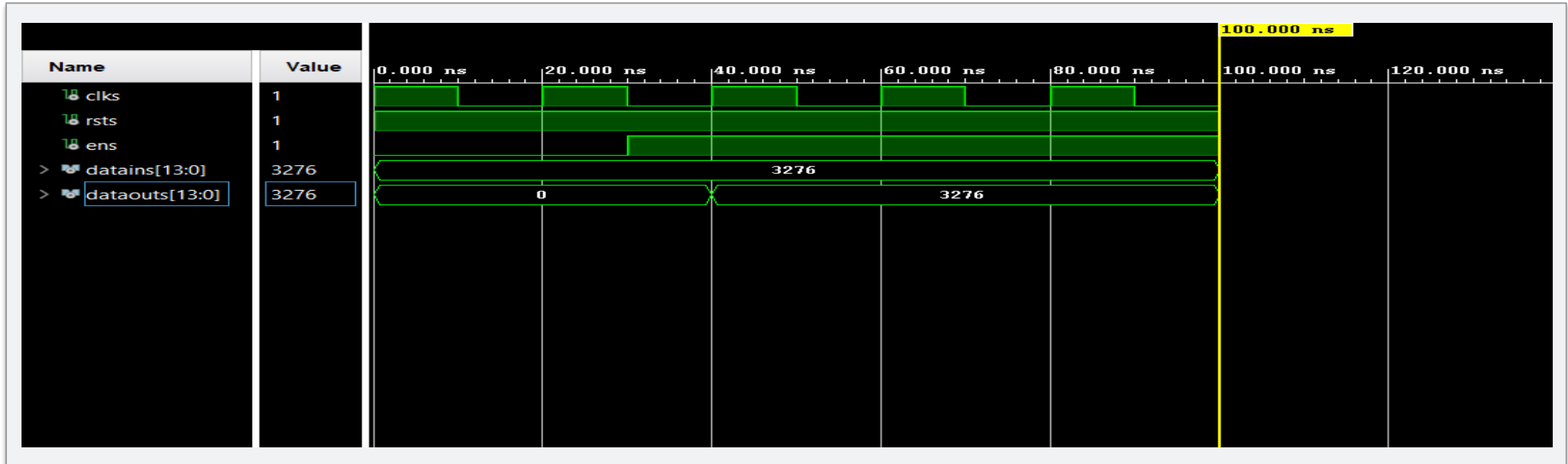
Reset & Width

$rst='0'$ → synchronous active-low clear ($reg \leftarrow 0$)
 $rst='1'$ → normal operation

Width = $Nb + \log_2 NFIFO = 14$ bit for $NFIFO=64$
Max value = $NFIFO \times 255 = 16320 \rightarrow$ fits in 14 bit

Accumulator - Testbench

Condition	datains (constant)	dataouts – expected
$t < 30 \text{ ns}$ (en='0')	3276 ("00110011001100")	0 - register not loaded yet
$t \geq \sim 40 \text{ ns}$ (en='1')	3276 ("00110011001100")	3276 — PIP0 register latches input



divide_by - Design

```
entity divide_by is
  generic (Nb: integer; log2NFIFO: integer);
  port (
    en, rst, clk : in std_logic;
    datain : in signed(Nb+log2NFIFO-1 downto 0);
    dataout : out signed(Nb+log2NFIFO-1 downto 0));
end divide_by;

architecture Behavioral of divide_by is
  constant Nacc : integer := Nb + log2NFIFO;
  signal reg : signed(Nacc-1 downto 0)
    := (others => '0');
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '0' then
        reg <= (others => '0');
      elsif en = '1' then
        reg(Nacc-log2NFIFO-1 downto 0) <=
          datain(Nacc-1 downto log2NFIFO);
        reg(Nacc-1 downto Nacc-log2NFIFO) <=
          (others => '0');
      end if;
    end if;
  end process;
  dataout <= reg;
end Behavioral;
```

Principle - Right Shift

$NFIFO = 2^{\log_2 NFIFO} \rightarrow$ division by NFIFO = arithmetic right-shift by $\log_2 NFIFO$ positions.

No multiplier or divider needed — implemented as pure wiring in hardware.

Bit Mapping (NFIFO=64, Nb=8)

datain [13:0] - accumulator output y_n
bits [13:6] $\rightarrow$ integer quotient
bits [5:0] $\rightarrow$ fractional (discarded)

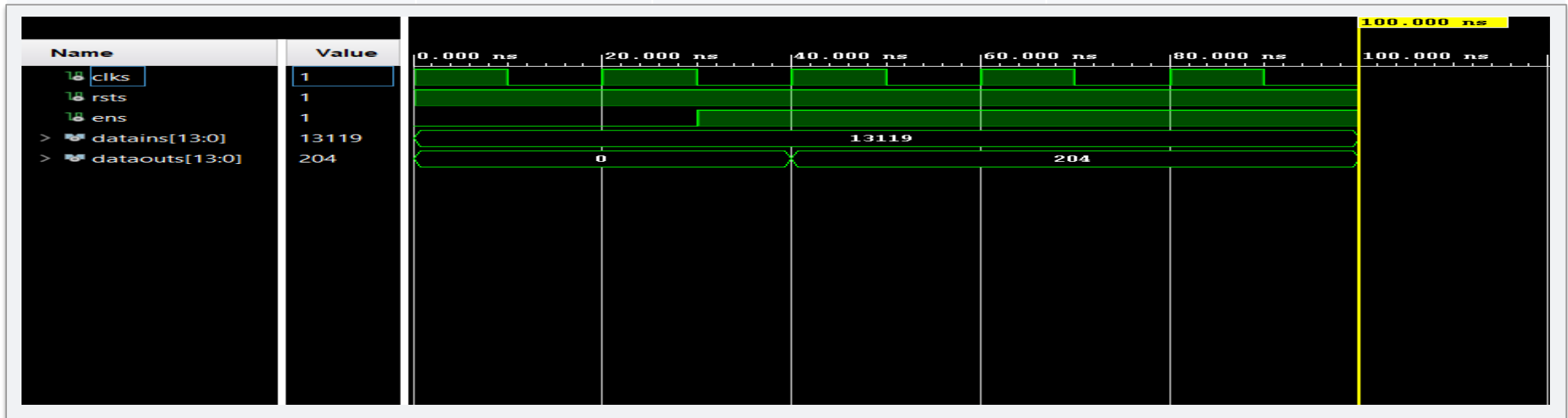
dataout [13:0]
bits [7:0] $\rightarrow$ result (0...255)
bits [13:8] $\rightarrow$ '000000' (zero-filled)

Example

datain = 13119 ("1100110011111")
 $13119 \div 64 = 204 \rightarrow$ dataout = 204
(Hex check: 333f $\rightarrow$ 00cc visible in Vivado Objects)

divide_by - Testbench

Signal	Value (dec)	Binary (14 bit)	Note
datains (constant input)	13119	11001100111111	Constant test vector
dataouts (expected)	204	00000011001100	$13119 \gg 6 = 204$
Hex visible in Objects panel	datains=333f	dataouts=00cc	Correct before waveform run



Frequency Response - NFIFO = 32, 64, 128, 256

NFIFO=32

$f_{\text{cutoff}} = f_s / (2 * \text{NFIFO}) = 781 \text{ kHz}$
Wider passband, less noise rejection

NFIFO=64

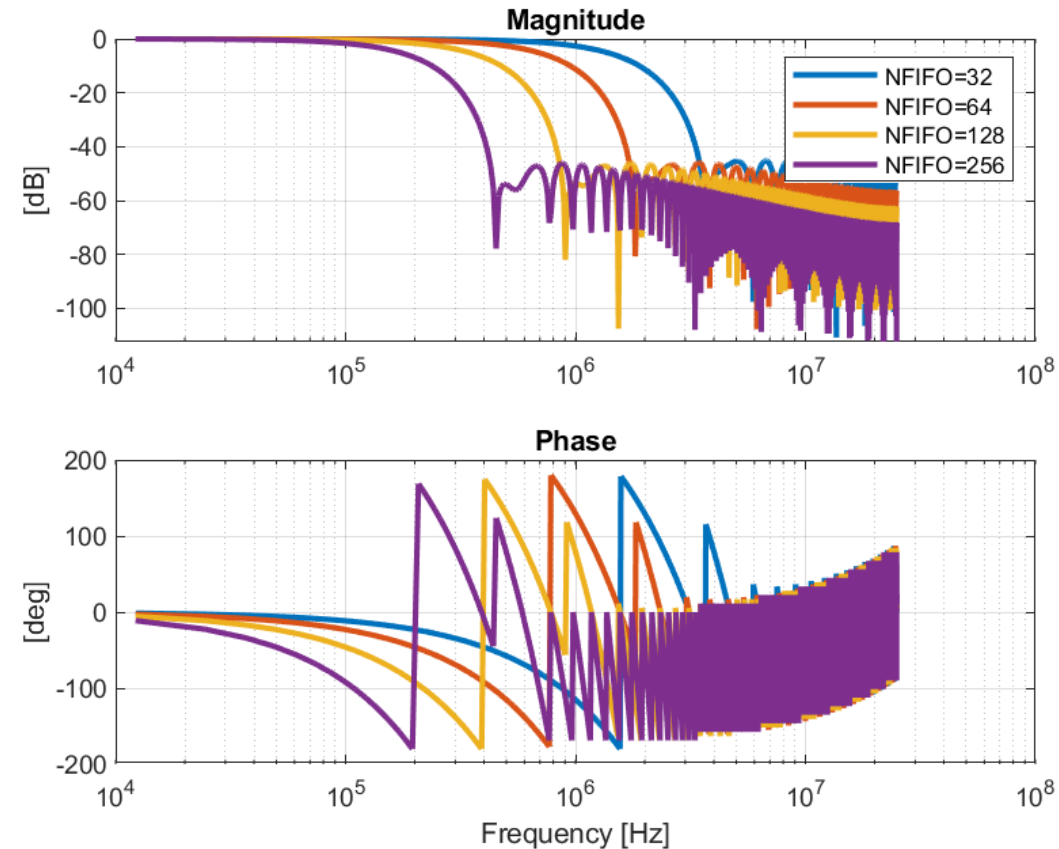
$f_{\text{cutoff}} = f_s / (2 * \text{NFIFO}) = 391 \text{ kHz}$
Good balance: signal + noise well separated

NFIFO=128

$f_{\text{cutoff}} = f_s / (2 * \text{NFIFO}) = 195 \text{ kHz} \approx f_0$
Narrowest band. Signal starts to attenuate!

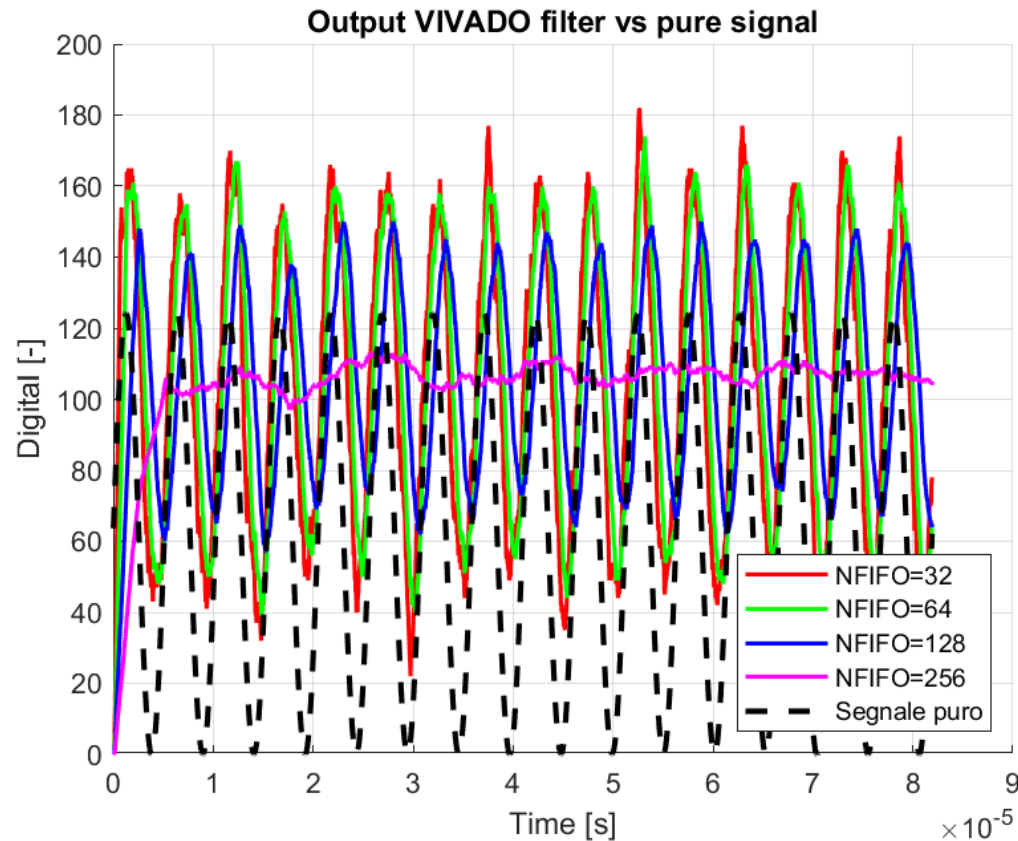
NFIFO=256

$f_{\text{cutoff}} = f_s / (2 * \text{NFIFO}) = 98 \text{ kHz} \approx f_0$
Band now below f_0 , signal itself is filtered out



Key: NFIFO=256 pushes f_{cutoff} below f_0 , the magnitude plot shows the signal frequency itself now lies in the stop-band.

Time Domain - ysignal + ynoise (filter output)



NFIFO=32/64/128 preserve the sinusoidal shape, NFIFO=256 collapses to a near-flat line, over-filtering.

NFIFO=32

More residual noise. 16-sample delay (0.32 μ s). Passes more noise.

NFIFO=64

Good balance. Noise clearly reduced, sinusoid well preserved.

NFIFO=128

Smoothest but 64-sample delay (1.28 μ s). Signal slightly attenuated at f_0 .

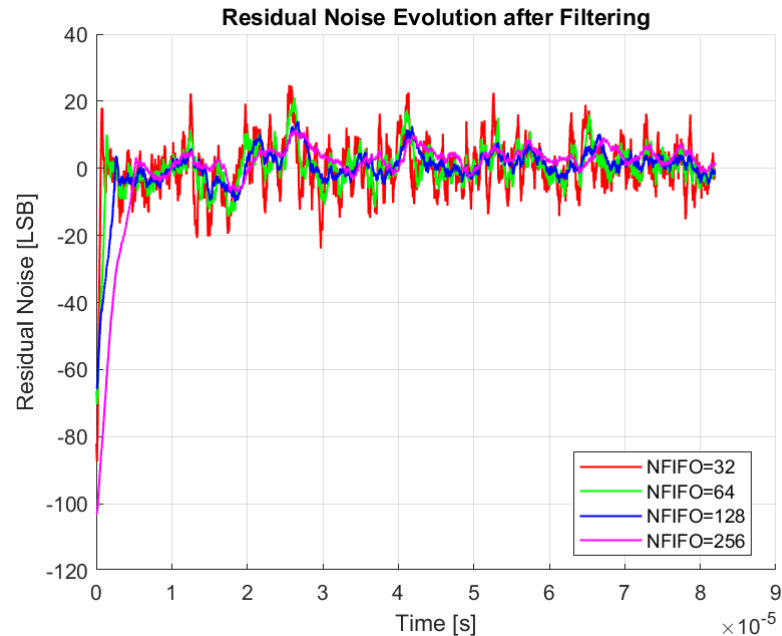
NFIFO=256

Signal nearly flat. Filter has removed the sinusoid itself, not just noise.

Pure signal

Lower DC: excludes noise mean offset (pnoise $\approx$ 44 LSB). Reference only.

Time Domain - ynoise (Residual Noise Evolution)



NFIFO = 32/64/128

Residual noise clearly visible, amplitude shrinks as NFIFO grows, expected behavior.

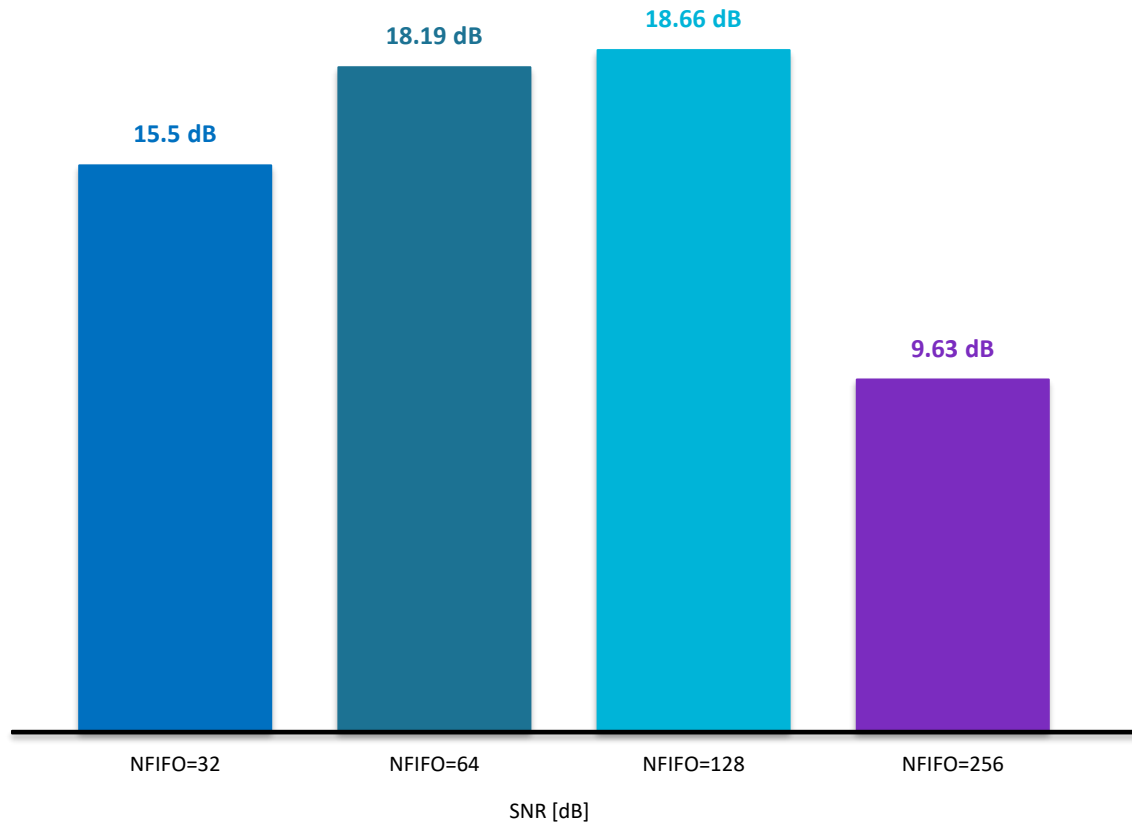
NFIFO = 256

Residual looks small, but this is misleading: the signal itself was removed, not just noise.

NFIFO	Noise RMS [LSB]	Reduction vs input	Noise Bandwidth
32	6.90	$\approx 3.4x$	781.25 kHz
64	4.79	$\approx 4.9x$	390.63 kHz
128	3.35	$\approx 7.0x$	195.31 kHz
256	4.26	$\approx 5.5x$	97.66 kHz

Reduction factor worsens from NFIFO=128 ($\approx 7.0x$) to NFIFO=256 ($\approx 5.5x$), confirms over-filtering, not improvement

Signal to Noise Ratio — NFIFO = 32, 64, 128, 256



NFIFO	fcutoff [kHz]	Noise RMS [LSB]	SNR [dB]	ΔSNR
32	781.25	6.90	15.50	—
64	390.63	4.79	18.19	+2.69
128	195.31	3.35	18.66	+3.16
256	97.66	4.26	9.63	-5.87

- SNR rises 32→64→128 (15.50→18.19→18.66 dB), then collapses at 256 (9.63 dB).
- Peak SNR occurs near NFIFO=128, where fcutoff $\approx f_o$, the optimal trade-off point.
- Beyond that point (NFIFO=256), fcutoff $< f_o$: noise keeps dropping but signal loss dominates, so SNR falls.
- Confirms SNR vs NFIFO is non-monotonic, a result not explored in NFIFO=32/64/128-only analyses.

Summary Table and Comments

Parameter	NFIFO = 32	NFIFO = 64	NFIFO = 128	NFIFO = 256
$\log_2$ NFIFO	5	6	7	8
f _{cutoff} [kHz]	781.25	390.63	195.31	97.66
Noise Bandwidth	Wide	Medium	Narrow	Below f_0
Noise RMS [LSB]	6.90	4.79	3.35 ← best	4.26
SNR [dB]	15.50	18.19	18.66 ← peak	9.63 ← worst
Δ SNR vs NFIFO=32	-	+2.69 dB	+3.16 dB	-5.87 dB
Group delay [samples] (theory / measured)	16 / 16.5	32 / 32.9	64 / 65.4	128 / 117.8 ← worst
Group delay [μ s]	0.32	0.64	1.28	2.56 ← worst
Signal atten. at f_0	None	Negligible	Significant	Severe
dataout width [bit]	13	14	15	16
Best use case	Low latency	Balanced	Max noise reject	Not recommended

SNR is non-monotonic, peaking at NFIFO=128 then dropping sharply at NFIFO=256 (f_{cutoff} < f_0). Group delay was independently validated via phase analysis of the fundamental component (Fourier projection), measured values match theory within ~3% up to NFIFO=128.

Conclusions

- ❑ **Moving Average Filter successfully implemented on FPGA in VHDL**
 - ❑ All 4 components verified independently with dedicated testbenches and end-to-end with `tb_filter_ma`
- ❑ **SNR vs NFIFO is non-monotonic, confirmed by extending the analysis to NFIFO=256**
 - ❑ SNR rises from 32→64→128 (15.50→18.19→18.66 dB), peaking near NFIFO=128, then collapses to 9.63 dB at NFIFO=256
 - ❑ The peak occurs exactly where $f_{\text{cutoff}} \approx f_0$: beyond it, the filter removes signal faster than it removes noise
- ❑ **Key trade-off: noise rejection vs group delay vs signal attenuation**
 - ❑ NFIFO=128 gives the best SNR with acceptable delay, NFIFO=64 is the best low-latency compromise
- ❑ **Possible improvements:**
 - ❑ Weighted MA (Hann/Hamming window) for sharper frequency roll-off
 - ❑ Configurable NFIFO via register to switch filter bandwidth at runtime
 - ❑ Pipelined FIFO for higher throughput at the same clock frequency